

计算机系统实验报告

实验三：应用开发实验

姓 名： 韦畅

学 号： 2352628

指导教师： 郭玉臣

实验日期： 2026 年 7 月 4 日

目录

1	实验目的	3
2	实验环境	3
3	系统架构	4
3.1	整体架构	4
3.2	数据通路	4
3.3	与实验二的关系	5
4	硬件设计	5
4.1	方向按钮接入 GPIO	5
4.2	七段数码管显示分数	6
4.3	不用 VGA	6
5	软件设计	6
5.1	新增原始字节发送 <code>uart_putc_raw</code>	7
5.2	贪吃蛇游戏逻辑	7
5.3	UART 协议帧	8
5.4	交叉编译	8
6	PC 端设计	9
6.1	协议解析与帧同步	9
6.2	多线程渲染架构	9
6.3	调试工具与启动器	10
7	下板验证	10
7.1	编译与烧录流程	10
7.2	验证结果	10
7.3	验证结论	15

8 总结	15
8.1 实验收获	15
8.2 遇到的问题与解决方法	16
8.3 可改进方向	16

1 实验目的

本实验在实验二已成功将 $\mu\text{C}/\text{OS-II}$ 移植到 OpenMIPS 处理器的基础上，进行桌面应用开发。具体目标如下：

1. 在 OpenMIPS + $\mu\text{C}/\text{OS-II}$ SoC 平台上开发一个完整的交互式桌面应用——**贪吃蛇游戏**，巩固在实时操作系统上编写用户任务的能力。
2. 设计 FPGA 与 PC 之间的 **UART 通信协议**：FPGA 端运行游戏逻辑并通过串口把游戏状态（蛇身、食物、分数）发送给 PC，PC 端解析后用 **pygame** 渲染图形界面，理解嵌入式系统中”主机—外设”的协同分工。
3. 掌握 **MIPS 交叉编译工具链**的获取与使用，完成 $\mu\text{C}/\text{OS-II}$ 内核与用户程序的编译、链接，生成可烧录的 **OS.bin**。
4. 通过对硬件约束（GPIO 按钮引脚、七段数码管）和软件逻辑的协同修改，复习并串联**计算机组成原理**（CPU、总线、外设）、**操作系统**（任务调度、时钟节拍）与**编译原理**（交叉编译、链接脚本）等知识。

与学长参考实现（VGA 翻牌游戏）相比，本实验在显示方式上做了改进：**不使用板载 VGA 硬件**，而是将画面渲染放在 PC 端用 pygame 图形界面显示，FPGA 仅作为”游戏主机”负责游戏逻辑运算与状态上报，更贴近真实嵌入式系统”瘦主机 + 胖客户端”的架构。

2 实验环境

项目	说明
FPGA 开发板	Nexys 4 DDR（Artix-7 100T，xc7a100tcsg324-1）
板载 Flash	Spansion S25FL128S（128Mbit Quad-SPI）
板载 USB-UART	FTDR 桥，PC 枚举为 COM3（USB Serial Port）
EDA 工具	Vivado 2024.2
交叉编译器	mips-sde-elf-gcc 5.3.0（Sourcery CodeBench Lite 2016.05）
编译主机	WSL2 Ubuntu（64 位，需 libc6:i386 等 32 位兼容库）
PC 端运行时	Python 3.13 + pygame 2.6.1 + pyserial 3.5
串口通信参数	4800 bps ，8 数据位，无校验，1 停止位
板载按钮	BTNC（中/开始）、BTNU/BTND/BTNL/BTNR（上/下/左/右）

表 1: 实验环境

关于波特率 4800: openmips.h 中 IN_CLK 注释写为 100 MHz, 据此 uart_init 计算的分频系数为 $100\text{M}/(16 \times 9600) \approx 651$; 但 OpenMIPS 实际运行在 clk_wiz 输出的 **50 MHz** 时钟上, 故实际波特率为 $50\text{M}/(16 \times 651) \approx 4800\text{ bps}$ 。PC 端串口必须匹配 4800 才能正确收帧 (与实验二 SuperCom 的 4800 配置一致)。

3 系统架构

3.1 整体架构

本系统采用”瘦 FPGA 主机 + 胖 PC 客户端”的分工: 游戏逻辑全部在 **FPGA** 内的 **OpenMIPS** 处理器上运行, PC 仅负责接收状态、渲染画面。两者通过 UART 串口单向通信 (FPGA → PC), 输入由开发板按钮完成, 不占用串口。系统结构如图 1 所示。

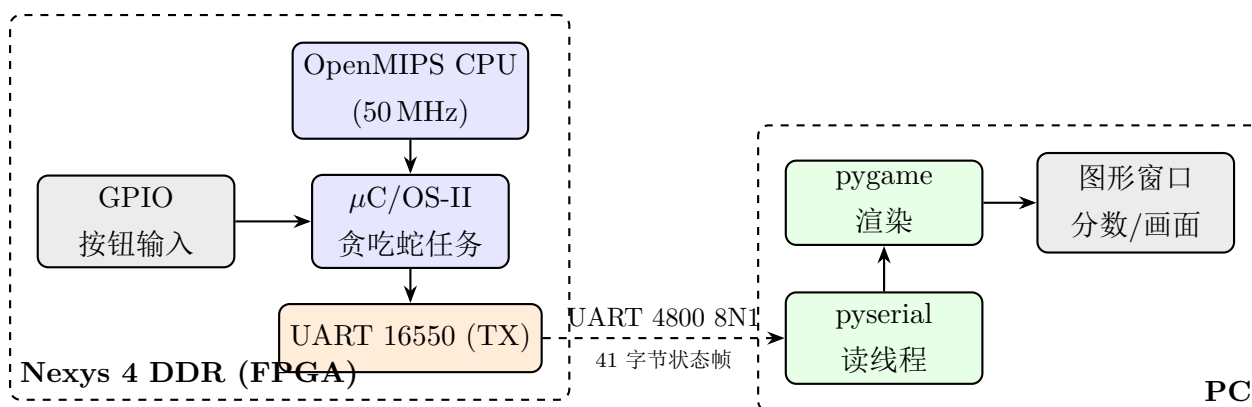


图 1: 贪吃蛇系统整体架构

3.2 数据通路

1. **输入:** 玩家按下开发板方向按钮 (BTNU/D/L/R), 电平送入 GPIO 输入寄存器, CPU 通过 `gpio_in()` 读取, 更新蛇的移动方向 (禁止 180° 反向)。
2. **游戏逻辑:** $\mu\text{C}/\text{OS-II}$ 的 `TaskStart` 任务每个时钟节拍 (`OSTimeDly(20)`, 约 200 ms) 执行一次: 蛇按当前方向前进一格, 判定吃食物/撞墙/撞自身, 更新蛇身坐标与分数。
3. **状态上报:** 每次移动后把蛇身位图、食物坐标、分数、游戏状态打包成固定 41 字节的二进制帧, 经 UART 发送给 PC。
4. **画面渲染:** PC 的 `pygame` 后台线程持续读串口、按帧头同步并校验, 主线程据最新帧重绘 16×16 网格画面。

3.3 与实验二的关系

本实验完整复用实验二的 SoC 硬件 (OpenMIPS 处理器、Wishbone 总线、UART16550、GPIO、SPI Flash、DDR2 MIG、 μ C/OS-II 操作系统与 BootLoader)，仅在以下三处做增量改动：(1) 约束文件把 4 个方向按钮接入 GPIO；(2) 加回七段数码管用于显示分数；(3) 替换用户任务 TaskStart 的函数体为贪吃蛇逻辑。地址映射、启动流程、交叉编译链均与实验二一致。

4 硬件设计

硬件以实验二工程为起点（整目录复制为 exp3，IP 重新生成），增量改动如下。

4.1 方向按钮接入 GPIO

实验二约束文件中 gpio_i[1..4] 原本接拨码开关，仅 gpio_i[0] 接了中心按钮 BTNC。为支持贪吃蛇四向控制，将 gpio_i[1..4] 改接 Nexys 4 DDR 的四个方向按钮（与软件按钮位定义对齐）：

GPIO 位	按钮	引脚	含义
gpio_i[0]	BTNC	N17	中（开始/重开）
gpio_i[1]	BTNU	M18	上
gpio_i[2]	BTNR	M17	右
gpio_i[3]	BTND	P18	下
gpio_i[4]	BTLN	P17	左

表 2: 方向按钮引脚约束

```

1 ## Direction buttons for snake: [1]=UP [2]=RIGHT [3]=DOWN [4]=LEFT
2 set_property -dict {PACKAGE_PIN M18 IOSTANDARD LVCMOS33} [get_ports {gpio_i
  [1]}]
3 set_property -dict {PACKAGE_PIN M17 IOSTANDARD LVCMOS33} [get_ports {gpio_i
  [2]}]
4 set_property -dict {PACKAGE_PIN P18 IOSTANDARD LVCMOS33} [get_ports {gpio_i
  [3]}]
5 set_property -dict {PACKAGE_PIN P17 IOSTANDARD LVCMOS33} [get_ports {gpio_i
  [4]}]

```

Listing 1: xdc 中方向按钮约束

顶层模块 `openmips_min_sopc` 的 `gpio_i[15:0]` 位宽不变，按钮电平由 `{15'h0, 1'b1, gpio_i}` 拼成 32 位送入 GPIO IP (bit16 恒为 1 用于 BootLoader 跳过 DDR2 初始化等待)。

4.2 七段数码管显示分数

实验二顶层为简化曾把数码管段选引脚注释掉（仅保留 8 位位选）。本实验为显示分数，重新接入数码管动态扫描模块 `seg7x16`：将其复制进工程，在顶层实例化，数据端接 GPIO 输出的高 16 位，软件写 `gpio_out(score<<16)` 即可让数码管显示分数（十六进制）。

```

1 // 端口：删掉原 gpio_o[7:0]，改为：
2 output wire [7:0] o_seg,
3 output wire [7:0] o_sel,
4 ...
5 // 内部：gpio_top 的 32 位输出 gpio_o_temp 高 16 位送数码管
6 seg7x16 seg7x16_inst(
7     .clk(clk_in), .reset(~rst_n), .cs(1'b1),
8     .i_data({16'b0, gpio_o_temp[31:16]}),
9     .o_seg(o_seg), .o_sel(o_sel)
10 );

```

Listing 2: 顶层加回 `seg7x16` 实例 (`openmips_min_sopc.v`)

约束文件按 Nexys 4 DDR 数码管引脚定义 `o_seg`（段选 CA–CG/DP）与 `o_sel`（位选 AN0–AN7）。

4.3 不用 VGA

与学长实现不同，本实验不使用 VGA 显示，因此不引入 `vga_display` 模块，省去 VGA 时序与电阻串 DAC 的复杂度。画面渲染完全由 PC 端 `pygame` 完成，FPGA 仅经 UART 上报状态。

5 软件设计

软件完整复用学长 操作系统和用户程序/ 目录 (BootLoader、 μ C/OS-II 内核、移植层、Makefile、链接脚本)，唯一实质改动在 HAL 文件 `ucosii/common/openmips.c`：(1) 新增 `uart_putc_raw`；(2) 把 `TaskStart` 函数体替换为贪吃蛇逻辑。`main` 函数 (`OSInit` $\rightarrow$ `uart_init` $\rightarrow$ `gpio_init` $\rightarrow$ `OSTaskCreate` $\rightarrow$ `OSStart`) 保持不变。

5.1 新增原始字节发送 uart_putc_raw

原 `uart_putc` 在发送 `0x0A` (`\n`) 时会自动追加 `0x0D` (`\r`)，这本是为文本换行设计的友好行为，但会破坏二进制协议帧的对齐（位图字节中可能恰好出现 `0x0A`）。为此新增一个不做任何字符替换的原始发送函数，专用于协议帧：

```
1 void uart_putc_raw(char c) {
2     unsigned char lsr;
3     WAIT_FOR_THRE;
4     REG8(UART_BASE + UART_TH_REG) = c;
5     WAIT_FOR_XMITR;
6 }
```

Listing 3: 原始字节发送 (openmips.c)

5.2 贪吃蛇游戏逻辑

游戏在 16×16 网格上进行。蛇身用文件级静态数组 `sx[]/sy[]` 存储坐标（下标 0 为蛇头），最大长度 256。游戏状态机有四态：`READY`（等待开始）、`RUNNING`（运行）、`GAMEOVER`（撞墙或撞自身）、`WIN`（蛇填满全屏）。由于任务栈仅 1KB，2KB 的蛇身数组必须放在静态全局区而非任务局部变量。

```
1 void TaskStart(void *pdata) {
2     OSInitTick();           /* 启动 100Hz 时钟节拍 */
3     snake_init();
4     while (1) {
5         if (state == ST_RUNNING) {
6             INT32U b = gpio_in();
7             /* 读方向按钮，禁止 180 反向 */
8             if ((b & BTN_UP) && dir != DIR_DOWN) dir = DIR_UP;
9             else if ((b & BTN_DOWN) && dir != DIR_UP) dir = DIR_DOWN;
10            else if ((b & BTN_LEFT) && dir != DIR_RIGHT) dir = DIR_LEFT;
11            else if ((b & BTN_RIGHT) && dir != DIR_LEFT) dir = DIR_RIGHT;
12            step_snake();      /* 前进 + 碰撞/吃食判定 */
13            send_frame();     /* 发状态帧给 PC */
14        } else {
15            send_frame();
16            if (gpio_in() & BTN_CENTER) { snake_init(); state = ST_RUNNING; }
17        }
18        OSTimeDly(SPEED);    /* SPEED=20 -> 200ms/格 */
19    }
20 }
```


Listing 4: TaskStart 主循环（节选）

回环边界：与经典贪吃蛇”撞墙即死”不同，本实现采用**回环边界**——蛇从一侧出去后从对侧回来（如左边界出则右边界进），降低难度、提升连贯性：

```
1 if (nx < 0) nx = GW - 1; else if (nx >= GW) nx = 0;
2 if (ny < 0) ny = GW - 1; else if (ny >= GW) ny = 0;
```

Listing 5: 回环边界处理（step_snake）

撞自身（蛇头命中除尾节外的身体）仍判 GAMEOVER；吃到食物则蛇身增长、分数 +1 并用 LCG 伪随机算法在空格上重新投放食物。

5.3 UART 协议帧

每次移动打包一个固定 41 字节的二进制帧发送，全量携带蛇身位图，丢帧不影响（下帧覆盖），无需应答。帧格式如表 3。

表 3: UART 协议帧格式（41 字节，FPGA → PC）

偏移	字段	字节	说明
0-1	HEAD	2	帧头 0xAA 0x55
2	state	1	3=READY / 0=RUNNING / 1=GAMEOVER / 2=WIN
3-4	score	2	分数，小端
5	food	1	食物坐标 (fx<<4) fy
6	head	1	蛇头坐标 (hx<<4) hy
7	length	1	蛇身长度（含头）
8-39	bitmap	32	16×16 位图，蛇身格 =1，行优先、 高位在前
40	checksum	1	bytes[2..39] 逐字节异或

发送时逐字节调用 `uart_putc_raw`（绝不能用 `uart_putc`，否则位图中的 0x0A 会引发帧错位）。41 字节/帧在 4800 bps 下约耗时 86 ms，200 ms 的移动节拍足以发完。

5.4 交叉编译

因本机与课程均未提供 MIPS 工具链，从 GitHub RT-Thread/toolchains-ci 下载 `mips-2016.05-7-mips-sde-elf-i686-pc-linux-gnu` 工具链安装到 WSL Ubuntu。该

工具链 GCC 版本 (5.3) 较学长使用的 4.3 新, 故不能复用学长预编译的 .o, 必须 make clean 后从源码全量重编。编译流程封装在 software/build.sh, 最终由 BinMerge.exe 合成 OS.bin (约 32KB)。

6 PC 端设计

PC 端用 Python + pygame 实现, 包含一个调试脚本和一个图形界面, 以及自动检测串口的启动器。

6.1 协议解析与帧同步

串口接收采用”搜帧头 + 定长读取 + 校验”的经典策略: 在接收缓冲里查找 0xAA 0x55, 找到后尝试读取 41 字节, 校验 XOR checksum 通过才视为有效帧。这种自同步方式对串口乱序、丢字节都具有鲁棒性。

```

1 def parse(frame):
2     chk = 0
3     for b in frame[2:40]:
4         chk ^= b
5     if chk != frame[40]:
6         return None          # 校验失败, 丢弃
7     state = frame[2]
8     score = frame[3] | (frame[4] << 8)
9     fx, fy = frame[5] >> 4, frame[5] & 0xF
10    hx, hy = frame[6] >> 4, frame[6] & 0xF
11    length = frame[7]
12    return state, score, fx, fy, hx, hy, length, bytes(frame[8:40])

```

Listing 6: 帧解析 (snake_gui.py)

6.2 多线程渲染架构

为避免串口读取阻塞画面, 采用后台读线程 + 主渲染线程结构: 后台线程持续解析串口并把最新帧放入队列, 主线程以 30FPS 轮询队列、重绘画面。位图按行优先高位在前的规则还原 16×16 网格, 蛇身画绿色方块、蛇头 (由 head 字段指示) 高亮深绿、食物画红色圆角块, 顶栏显示分数与游戏状态。

```

1 for y in range(GRID):
2     for x in range(GRID):
3         idx = y * GRID + x
4         bit = (bitmap[idx >> 3] >> (7 - (idx & 7))) & 1

```

```

5         if bit:
6             color = (90,255,90) if (x==hx and y==hy) else (40,170,40)
7             pygame.draw.rect(screen, color, rect)

```

Listing 7: 主渲染循环（节选）

6.3 调试工具与启动器

debug_rx.py 把接收到的每帧解析后以 ASCII 字符画打印（@@ 蛇头、[] 蛇身、** 食物、. 空），用于在 GUI 之前单独验证 FPGA 发帧与协议正确性。

play.py 是启动器，用 pyserial 枚举串口、自动挑出板子（描述含“USB Serial Port”/“Digilent”等、排除蓝牙），免去手记 COM 号；支持 --debug 切换调试模式，并提供 play.bat（Windows 双击）与 play.sh（Git Bash）两种入口。

7 下板验证

7.1 编译与烧录流程

1. 软件编译：在 WSL 中执行 `bash software/build.sh`，依次编译 BootLoader 与 $\mu\text{C}/\text{OS-II}$ + 用户程序，链接生成 `ucosii.bin`，再由 `BinMerge.exe` 合成最终 `OS.bin`。
2. 硬件综合：Vivado 2024.2 打开 `exp3.xpr`，重新生成 IP（MIG、clk_wiz），依次 Run Synthesis → Run Implementation → Generate Bitstream，得到 `openmips_min_sopc.bit`。
3. 烧录：Hardware Manager 连接开发板，Add Configuration Memory Device 选 `s25f1128sxxxxx0-sp`（板载 Spansion Flash），把 `OS.bin` 写入（Erase + Program + Verify）；再 Program Device 下载比特流启动 FPGA。
4. 运行：PC 运行 `python play.py`，自动检测 COM3 启动 pygame。

7.2 验证结果

(1) 串口帧验证：先用 debug_rx.py 验证 FPGA 发帧与协议。按下中键前处于 READY 态，终端持续打印蛇身 ASCII 位图，蛇头（@@）、蛇身（[]）、食物（**）位置正确，如图 2。

[illegible]

图 2: 串口帧调试输出 (debug_rx.py)

(2) **pygame 游戏画面**: 进入 GUI 后按中键开始, 蛇以约 5 格/秒的速度移动, 方向按钮实时转向, 吃到食物变长、分数 +1。如图 3 为运行中画面。

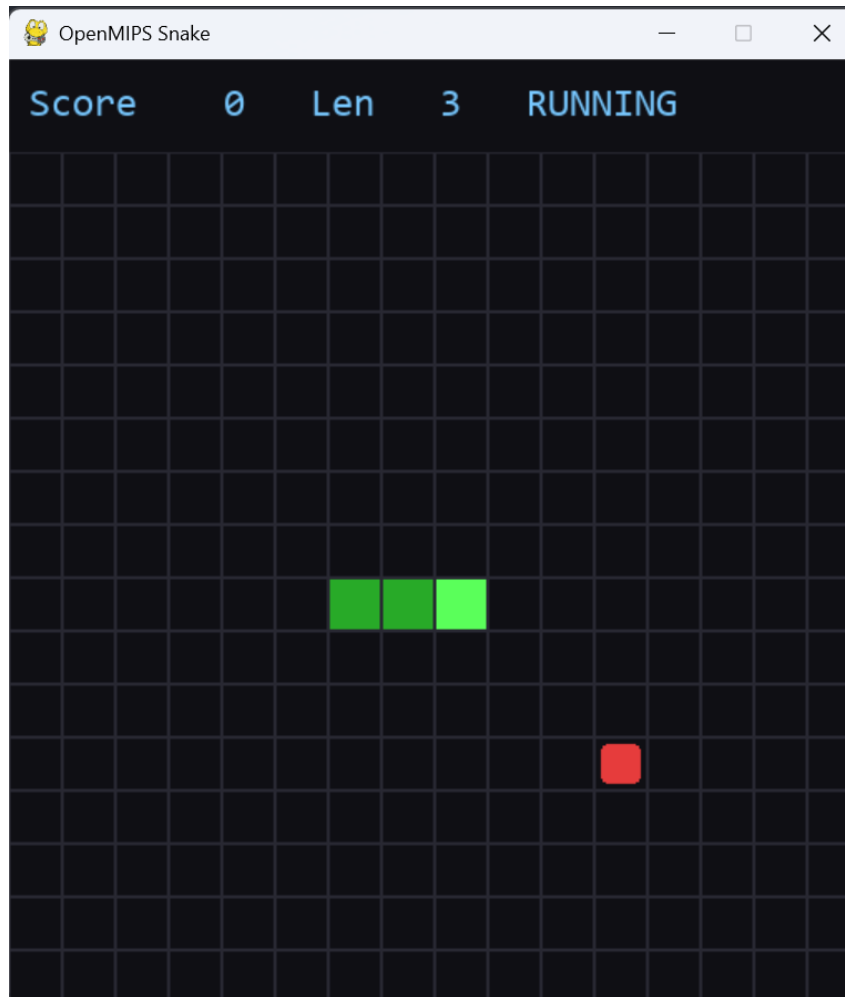


图 3: pygame 运行中画面

(3) **边界回环**: 蛇触及任一边界不再 Game Over，而是从对侧穿出（验证回环边界逻辑）。

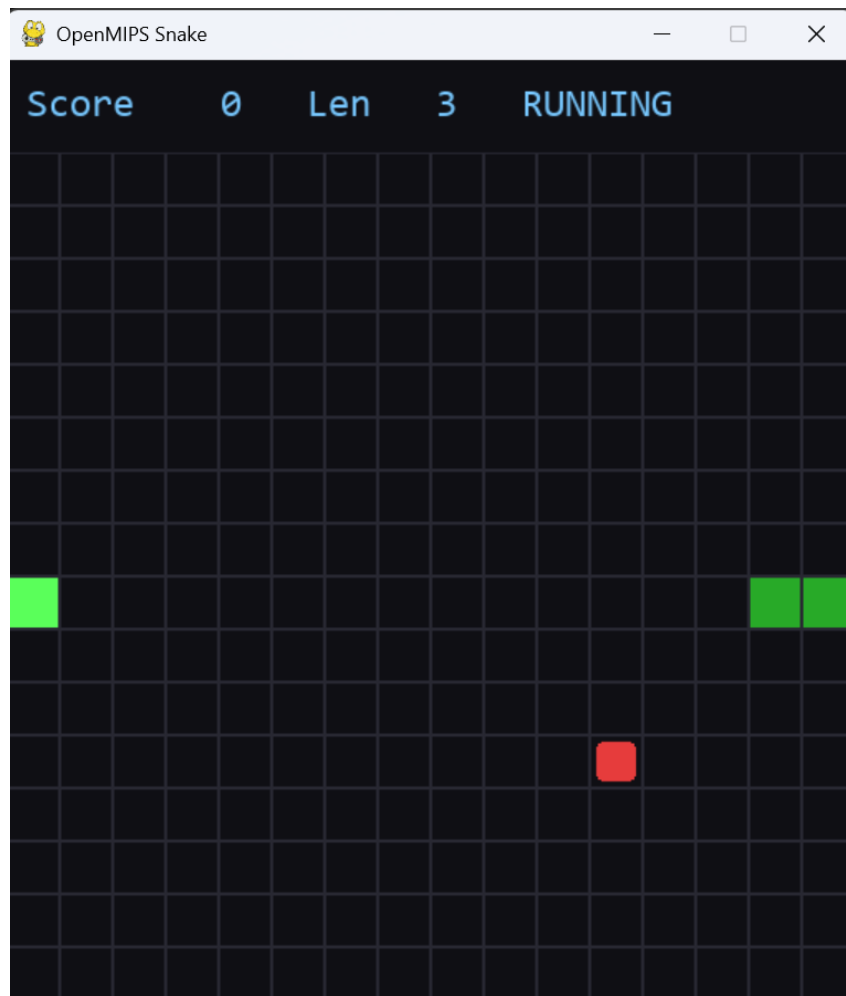


图 4: 边界回环：蛇头从一侧穿出，从对侧穿进

(4) **Game Over 与重开**：蛇头撞到自身时进入 GAME OVER 态，画面停止；按中键（BTNC）重开新局。

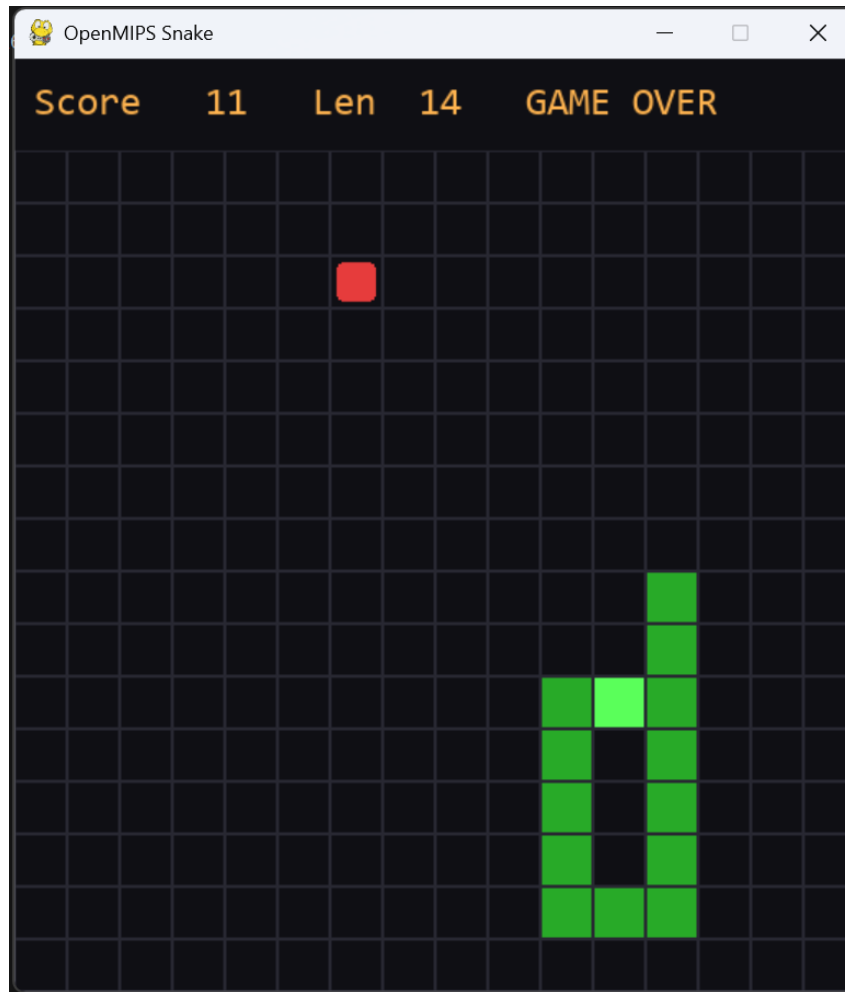


图 5: Game Over 画面

(5) 数码管分数：板上七段数码管同步显示分数（十六进制），随吃食递增，如图 6。

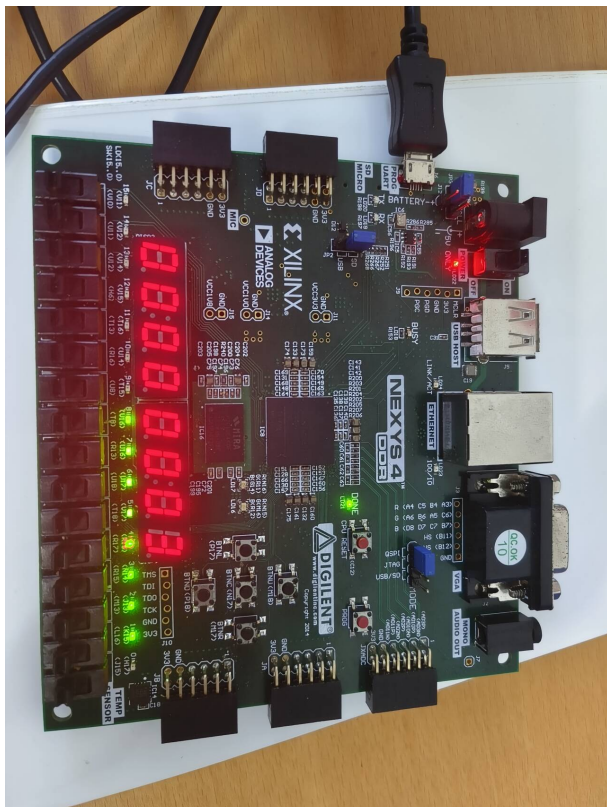


图 6: 数码管显示分数（开发板实物）

7.3 验证结论

板载按钮（中/上/下/左/右）全部生效，UART 通信稳定，画面渲染流畅，数码管同步显示分数，边界回环与速度减半均符合预期。全部功能验证通过。

8 总结

8.1 实验收获

通过本次贪吃蛇应用开发，我对以下内容有了更深入的理解：

1. 实时操作系统应用开发：在 $\mu C/OS-II$ 上编写用户任务，理解了任务调度、时钟节拍（CP0 Count/Compare）、OSTimeDly 节奏控制的机制。
2. 通信协议设计：从零设计了一个鲁棒的二进制帧协议（帧头定界 + 定长 + 位图 + XOR 校验），体会到“自同步”对串口丢字节的重要性。
3. 交叉编译全流程：独立获取并部署 MIPS 工具链，理解了链接脚本、入口地址、`-nostdlib` 下无标准库的约束，以及 GCC 版本差异导致必须全量重编的坑。

4. **软硬件协同**：通过 GPIO 同时驱动按钮输入与数码管输出、UART 上报状态，把 CPU、总线、外设、操作系统、应用串成完整链路。
5. **嵌入式调试方法**：先用 `debug_rx` 单独验证 FPGA 发帧，再接 GUI 渲染——分层隔离的调试思路极大提升了定位问题的效率。

8.2 遇到的问题与解决方法

1. **MIPS 工具链缺失**：本机与课程均未提供 `mips-sde-elf-gcc`。
解决：从 GitHub RT-Thread/toolchains-ci release v1.1 下载 Sourcery CodeBench 2016.05 工具链，在 WSL Ubuntu 安装 32 位兼容库（`libc6:i386` 等）后运行。
2. **UART 收不到数据**：PC 端最初按 9600 波特率接收，完全收不到帧。
解决：`openmips.h` 的 `IN_CLK=100MHz` 是误导注释，CPU 实际跑 50MHz，分频系数 651 对应实际波特率 4800。PC 端改 4800 后立即正常。
3. **Flash 烧录型号选错**：Vivado 默认候选 `mt25ql128`，烧录报 `Part selected mt25ql128, but part s25fl128sxxxxx0 detected`。
解决：本板 Flash 实为 Spansion S25FL128S，改选 `s25fl128sxxxxx0-spi-x1_x2_x4` 后烧录成功。
4. **搬工程后 Vivado Sources 为空**：复制实验二工程目录并改名后，打开 `exp3.xpr` 设计源面板无文件。
解决：原空白工程残留的 `exp3.cache/.hw/.sim/.ip_user_files` 与新 `.xpr` 冲突，删除这四个目录后重新打开即正常加载。
5. **二进制帧被字符过滤破坏**：原 `uart_putc` 把 `0x0A` 自动追加 `0x0D`，导致位图字节错位、PC 校验失败。
解决：新增 `uart_putc_raw` 原始字节发送函数，协议帧专用，杜绝字符替换。
6. **WSL 编译 PATH 报语法错**：WSL 继承的 Windows PATH 含 Program Files (x86)，括号使 `export PATH=...:$PATH` 报 bash 语法错误。
解决：编译时用临时 `PATH="/home/.../bin:/usr/bin:/bin"` 绕过，并封装进 `build.sh`。

8.3 可改进方向

- 数码管目前显示十六进制分数，可改为十进制 BCD 显示更直观。
- 可在画面上叠加游戏开始/结束的提示文字，或加入难度递增（速度随分数提升）。
- 协议帧可由“全量位图”优化为“增量帧”，降低 UART 流量，支持更高帧率。